1. CONTROL FLOW TESTING

1. AIM:

To demonstrate the control flow testing with different test case for sum of n even numbers.

2. ALGORITHM:

Step-1: Open C++ editor and write the program for sum of even numbers.

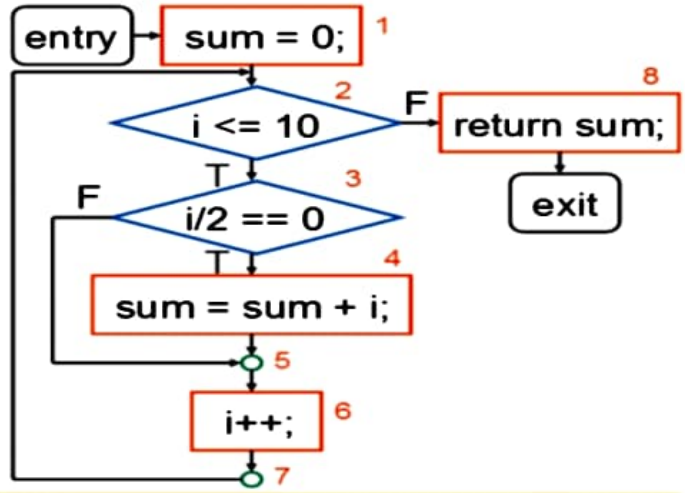
Step-2: Draw a flow diagram for the program which demonstrates the control flow.

Step-3: To ensure the statement, condition, decision and path coverage.

Step-4: To ensure the flow of the decisions based on the inputs.

Step-5: Write the test cases, it should ensure all the paths, conditions and decisions in this program are executed at least once.

3. FLOW DIAGRAM:



4. CODING:

```
#include<iostream.h>
#include<conio.h>
void main(){
clrscr();
int sum=0,i, j=60;
cout<<"Enter the I value :";
cin>>i;
while(i<=j){

if(i%2==0){
cout<<i<<"\n";
sum=sum+i;
}
i++;
}
cout<<"Result :"<<sum;
getch();
}
```

OUTPUT:

Enter the I value: 50

Result: 330

5. **TEST CASE:**

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Results	Actual Results	Pass/Fai
CFT01	Statement Coverage	Every statement in the program executed at least once.	1→2→3→4→5→→6→7→2→8 I=50, J=60, Sum=0	330	Error	Fail
CFT02	Decision Coverage	Every decision in the program has taken all possible outcomes at least once.	1→2→3→4→5→6→→7→2 3→4→5→6→7→2→8 Check 2→ True & False Check 3 →True & False I=50, J=60, Sum=0	330	Error	Fail
CFT03	Condition Coverage	Every Condition in each decision has taken all possible outcomes at least once.	I<=J I%2==0 I=50, J=60, Sum=0	330	Error	Fail
CFT04	Path Coverage		1→2→3→4→5→→6→7→2→8 I=50, J=60, Sum=0 I<=60, i%2==0, i+1>10	330	Error	Fail

2. DATA FLOW TESTING

1. AIM:

To demonstrate the data flow testing of one control structure with different test case for finding electricity bill.

2. ALGORITHM:

Step 1: Open C++ editor and write the EB bill calculation program.

Step 2: Draw a flow graph for the corresponding program.

Step 3: Compile and execute the program.

Step 4: To ensure the system is getting No. of input units from the user.

Step 5: To ensure the flow of the data (Input Flow) based on the input units.

Step 6: Compile and execute the program

Step 7: Display the result

3. CODING:

```
#include<iostream.h>
#include<conio.h>
void main()
{
inta,b,c,d,e,f,g,h,units;
clrscr();
cout<<"\n Electricity Bill Calculation";
cout<<"\n Enter the units consumed:";
cin>>units;
if(units<=100){
  cout<<"\n There is no charge";
}else if(units>100 && units <=200)  {
  a=units-100;
  b=a*1.5;
cout<<"\n Your EB Bill:"<<b;
}
else if(units>200 && units<=500){
  a=units-100;
```

2 program

```
  b=a-100;
  c=100*2;
  d=b*3;
  e=c+d;
  cout<<"\n Your EB bill :"<<e;
}else if(units>500){
  a=units-100;
  b=a-100;
  c=100*3.5;
  d=b*4.6;
  e=a-b;
  f=e*6.6;
  h=c+d+f;
  cout<<"\n Your EB bill:"<<h;
}else{
  cout<<"\n Error in units";
}
getch();
}
```

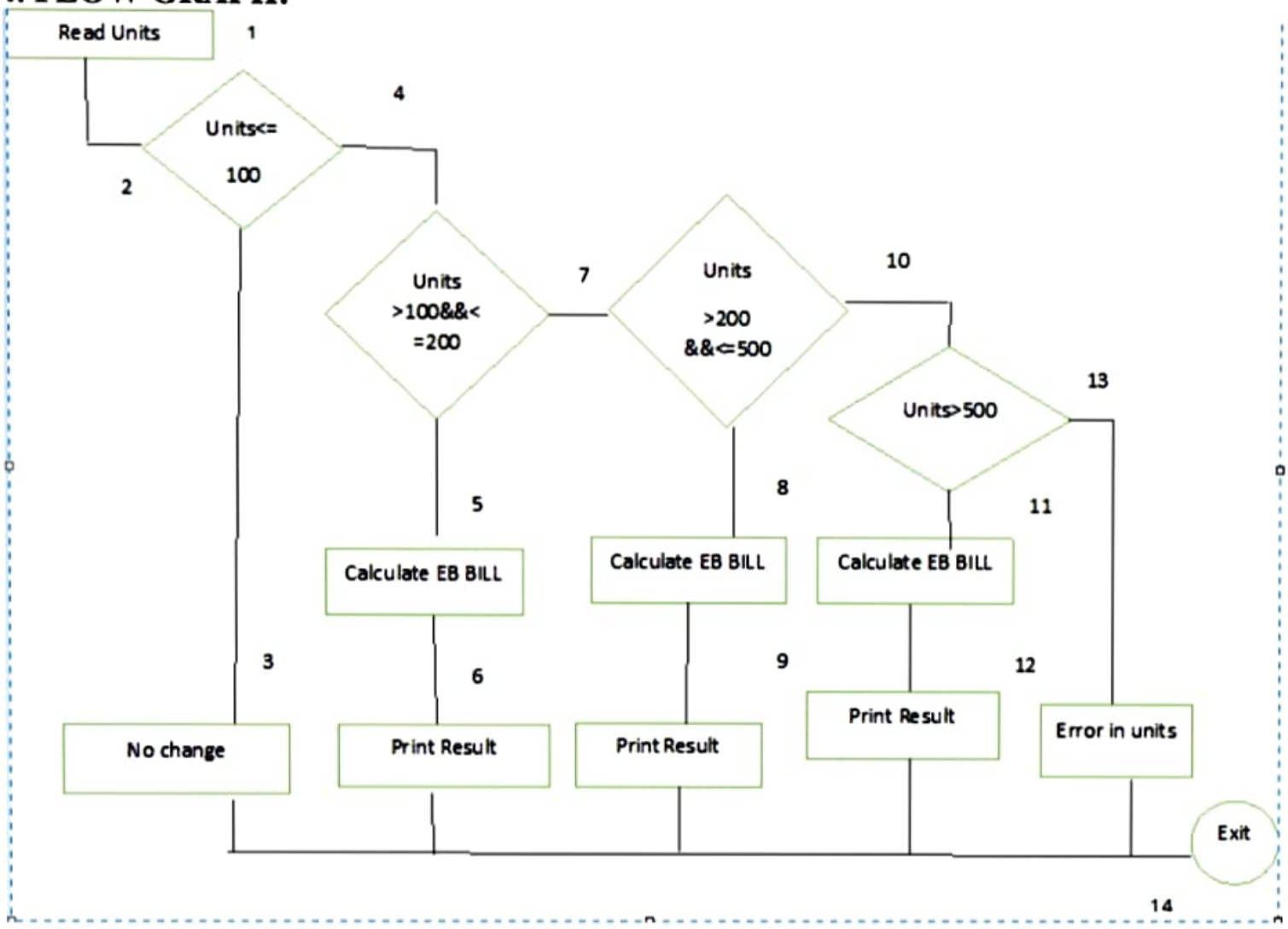
Output:

Electricity Bill Calculation

Enter the units consumed: 280

Your EB bill: 440

4. FLOW GRAPH:



5. TEST CASE

ID	Test scenario	Test step	Test data	Expected result	Actual result
1	Check the member Definition	To check all variables are defined and used.	The variables a,b,c,d,e,f,g and h	All the variables a,b,c,d,e,f,g and h should be defined and used.	Fail. The variable g is initialized but not used.
2	Check the Data flow	Sample inputs 1->2->3	Consumed units 75.5	Should be Display error	Fail. It accept the decimal input values even though the variable "units" initialized using int.
3	Check the Data flow	Calculation 1->4->5->6	Consumed units 150	EB Bill amount should be Rs. 75	Pass
4	Check the Data flow	Calculation 1->7->8->9	Consumed units 280	EB Bill amount should be Rs.440	Pass
5	Check the Data flow	Calculation 1->10->11->12	Consumed units 510	EB Bill amount should be Rs.1194	Pass

3. BRANCH TESTING

1. AIM

To demonstrate the branch testing with different test cases for calculating students attendance proforma.

2. DEFINITION

Branch coverage is a testing method which aims to ensure that each one of the positive branch in each decision point is executed at least once and thereby ensuring that all reachable code is executed ie., every branch takes away either true or false. Calculate branch coverage, one has to find out the minimum no. of paths which will ensure that all the edges are covered.

3. ALGORITHM

Step 1: Open C++ editor and type the attendance proforma code and save it.

Step 2: Draw a flow graph for the corresponding program.

Step 3 : Write the branch coverage test cases.

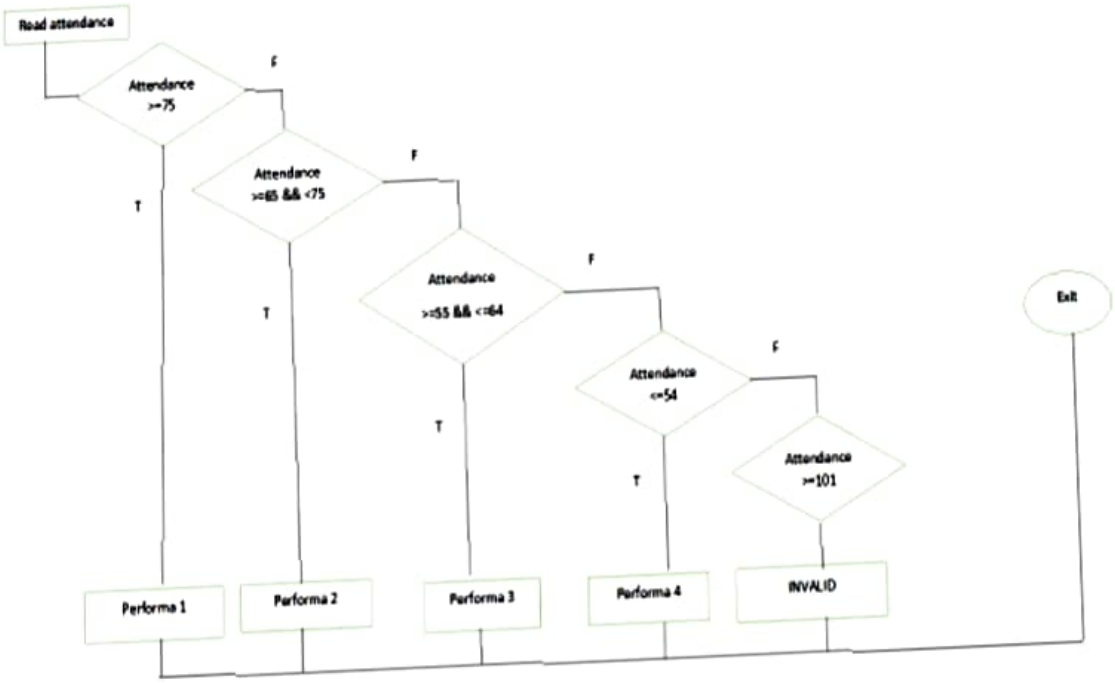
Step 3: Compile and execute the program based on the test case.

Step 4: To ensure the system is getting attendance percentage as input from the user.

Step 5: To ensure all the branches are covered based on the input in both true and false parts.

Step 6: Ensure the test case coverage and track the appropriate results.

4. FLOW DIAGRAM



5. CODING

3 Branch Testing

```
#include<iostream.h>
#include<conio.h>
void main(){
int attend;
clrscr();
cout<<"\n Enter Attendance:";
cin>>attend;

if(attend>=75  && attend<=100)      {
                cout<<"\nYou are in PROFORMA I ";
                cout<<"\nYou are eligible to write all the exams";
        }else if (attend>=65 && attend<=74)
        {
                cout<<"\n You are in PROFORMA II";
                cout<<"\n You are allowed to write the exam with the permission of the principal.";
        }else if (attend>=55  && attend<=64)
        {
                cout<<"\n You are in PROFORMA III";
                cout<<"\n You can't write current semester but you can write arrear exam";
        }else if (attend<=54)
        {
                cout<<"\n You are in PROFORMA IV";
                cout<<"\n Not continue to study need to rejoin the course in the next year";

        }else   {
                cout<<"\nInvalid input";
        }
getch();
}
```

Output:

Enter Attendance:55

You are in Performa-III

You can't write current semester but you can write arrear exam

6. TEST CASE

3. Branch Testing

ID	Test scenario	Test steps	Test data	Expected result	Actual result	Result
BT01	A>=75 &&<100	Covered Paths through Branches 1- 2- 3-11	Attendance = 99%	PROFORMA I	Yes	Pass
BT02	A>=65 &&<=75	Covered Paths through Branches 1- 2- 4-5-11	Attendance = 70%	PROFORMA II	Yes	Pass
BT03	A>55 &&<=64	Covered Paths through Branches 1-2-4-6-11	Attendance = 60%	PROFORMA III	Yes	Pass
BT04	A<=54	Covered Paths through Branches 1-2-4-6-8-9-11	Attendance = 48%	PROFORMA IV	Yes	Pass
BT05	A>100	Covered Paths through Branches 1-2-4-6-8-10-11	Attendance = 101%	INVALIDINPUT	Yes	Pass

Decision	Possible outcomes	Test case				
		BT01	BT02	BT03	BT04	BT05
Attendance >=75	T	X				
Attendance >65 &&<75	T		X			
Attendance >55 &&<64	T			X		
Attendance <=54	T				X	
Attendance > 100	T					X

4. Unit Testing

AIM: **To demonstrate the unit testing with a simple user registration form.**

Definition:

UNIT TESTING is a type of software testing where individual units or components of software are tested. The purpose is to validate that each unit of the software code performs as expected. Unit Testing is done during the development (coding phase) of an application by the developers.

ALGORITHM:

Step1: Design a simple user registration form using HTML.

Step2: Write a Javascript for validation in the same HTML page.

Step3: Call the Javascript function for validation when clicking the html submit button.

Step4: Write the test case for validating all the individual fields.

Step5: Test all the individual units; if there is no error, the unit testing is done successfully.

Step6: To ensure all the test cases should produce appropriate results.

PROGRAM

Validate.html

```
<html>
<head>
<title> Form validation</title>
<script type="text/javascript">
function validate() {
var uname = document.getElementById("user");
if(uname.value =="){
alert("USER NAME should not be an empty");
uname.focus();
return false;
}
var pass = document.getElementById("pass");
if(pass.value =="){
alert("PASSWORD should not be an empty");
```

```
pass.focus();
return false;
}
var repass = document.getElementById("repass");
if(repass.value =""){
alert("RE-PASSWORD Shouldnot be empty");
repass.focus();
return false;
}
if(pass.value!=repass.value){
alert("PASSWORD and RE-PASSWORD are not same.. Please give the Same");
repass.focus();
return false;
}
var email= document.getElementById("email");
if(email.value =""){
alert("Email Shouldnot be empty");
email.focus();
return false;
}
if(email.value.indexOf("@",0)<0){
alert("Email Should Contail @ symbol");
email.focus();
return false;
}
if(email.value.indexOf(".",0)<0){
alert("Email Should Contain . symbol");
email.focus();
```

```
return false;
}
var mobileno = document.getElementById("mobileno");
if(mobileno.value ==""){
alert("Mobile NO. Shouldnot be empty");
mobileno.focus();
return false;
}
if(isNaN(mobileno.value)){
alert("Mobile NO. Should be Numeric");
mobileno.focus();
return false;
}
if(mobileno.value.length<10){
alert("Mobile NO. Should be 10 digit only");
mobileno.focus();
return false;
}
if(mobileno.value.length>10){
alert("Mobile NO. Should be 10 digit only");
mobileno.focus();
return false;
}

}
</script>
</head>
<body bgcolor="pink">
<form method="post" action="val1.html" name="contact form">
```

4 Unit Testing (3)

```
<h1 align="center">USER REGISTRATION FORM</h1>
<table align="center"border="1">
<tr>
<td>Name:</td>
<td><input type="text" size="20" name="user" id="user"</td>
</tr>
<td>Pass:</td>
<td><input type="password" size="8" name="pass" </td>
</tr>
<tr>
<td>Repass:</td>
<td><input type="password" size="8" name="repass" </td>
</tr>
<tr>
<td>email:</td>
<td><input type="text" size="20" name="email" id="email"</td>
</tr>
<tr>
<td>Mobileno:</td>
<td><input type="text" size="10" name="mobileno" id="mobileno"</td>
</tr>
<tr>
<td><input type="submit" value="submit" name="submit" onclick="return validate();" <td>
<td><input type="reset" value="reset" name="reset"</td>
</tr>
</table>
</body>

</html>
```

Val1.html

```
<html>
<head>
</head>
<body>
Thank YOu for YOUr Registration
</body>
</html>
```

4 Unit Testing (5)

TEST CASE

ID	Test scenario	Test steps	Test data	Expected result	Actual result	Result
UT01	Submitting UR Form	Check User Name Field	empty	Should not empty	USER NAME should not be empty	Pass
UT02	Submitting UR Form	Check Password Field	empty	Should not empty	PASSWORD should not be empty	Pass
UT03	Submitting UR Form	Check RePass Field	empty	Should not empty	RE-PASSWORD should not be empty	Pass
UT04	Submitting UR Form	Check both Password are same	Abc Abc	If Password is Same should be Accepted	Password Accepted	Pass
UT05	Submitting UR Form	Check both Password are same	Abc xyz	If Password is Not Same should not be Accepted	PASSWORD and RE-PASSWORD are not same. Please give the same.	Pass